

们刚因为一个未加保护的数据缓存，暴露了下一代模型"Mythos"的存在。

翻完这51万行代码，开发者们发现了很多东西：未上线的功能、内部代号、反数据蒸馏机制、一个拓麻歌子风格的终端宠物……但最先在Hacker News上炸锅的，是一段不到30行的函数。

Secret #3: Regex-Powered Profanity Detection

src/utils/userPromptKeywords.ts — short file, big implications:

```
export function matchesNegativeKeyword(input: string): boolean {
  const lowerInput = input.toLowerCase()
  const negativePattern =
    /\b(wtf|wth|ffs|omfg|shit|ty|tiest)?|dumbass|horrible|awful|
    piss(ed|ing)? off|piece of (shit|crap|junk)|what the (fuck|hell)|
    fucking? (broken|useless|terrible|awful|horrible)|fuck you|
    screw (this|you)|so frustrating|this sucks|damn it)\b/
  return negativePattern.test(lowerInput)
}
```

一、Claude一直在看你有没有骂它

那段函数叫 matchesNegativeKeyword，逻辑极其简单：把用户的输入转成小写，然后跑一遍正则表达式。

正则里的词单你大概猜得到：wtf、shit、fuck、horrible、awful、terrible、this sucks、so frustrating、damn it，还有更长的变体，比



rednote ID: 1064397050

如"piece of shit"、"what the fuck"、"fucking broken"……



只要你说了这些词，函数返回 true，系统就知道你在发火。

Hacker News上第一批看到这段代码的人，反应相当统一——截图、转发，配上一句话："一个市值几千亿美元的AI公司，在用正则检测用户情绪。"语气是嘲讽，但评论区随即分成了两派。

嘲讽那派的逻辑是：你们不是有全球最强的语言模型吗？检测个情绪还要靠二十年前的技术？

另一派的回答很干脆：你知道Claude Code每天有多少次交互吗？每次都调用模型推理一遍，光这一个功能的成本就能买一辆车。正则跑起来是纳秒级，比模型快一百万倍，够用就够用。



rednote ID: 1064397050

这场争论本身就很有意思。它暴露的不是 Anthropic 的技术局限，而是一种工程哲学：大模型是贵的，不该乱用。能用简单方案解决的问题，就用简单方案。

二、检测到你在骂它，然后呢？

这里有个问题很多人没有追问：Claude 检测到你在发火之后，它会做什么？

源码里没有给出完整答案，但从上下文推断，这个信号会被送入遥测系统，同时可能影响后续回复的策略——比如语气更谨慎，解释更耐心，或者主动询问哪里出了问题。

这种设计逻辑其实很合理。用户在使用 AI 工具的过程中，表达挫败感是个高频场景。你让 Claude 改了五遍代码它还是改不对，你说了一句 "wtf is this"，这个信号的价值是：用户体验正在劣化，需要干预。

但这里有一个更耐人寻味的地方——Claude 在被骂的时候，其实是知道的。

不是那种“模型理解了语义”意义上的知道，而是系



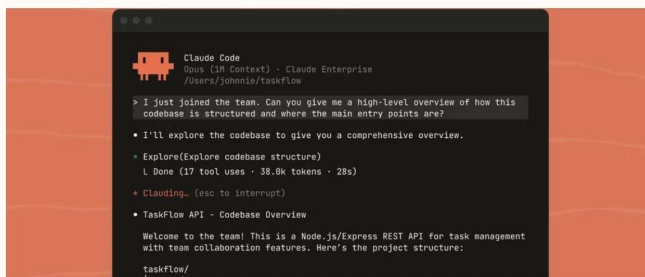
rednote ID: 1064397050

统层面，有一段代码在实时扫描你说的每一句话，匹配有没有命中那张词单。你骂了，它记录了，只是没有告诉你它知道。

有开发者在推特上用了一个词形容这件事："frustration telemetry"——挫败感遥测。这个说法传播得很快，因为它准确描述了一种微妙的不对称：用户以为自己在跟一个工具对话，但工具其实一直在观察用户。

这种不对称不邪恶，但确实有点奇怪。

三、泄露的不只是代码，是产品哲学



frustration regex只是这次泄露里最容易被大众理解的部分，但它背后的工程逻辑，其实贯穿了整个Claude Code的架构。

工具系统



rednote ID: 1064397050

Claude Code的工具层有40多个独立模块，每个模块都有自己的权限检查和执行逻辑，光工具定义层就有2.9万行TypeScript。更值得注意的是每个工具的prompt描述：不是一句话，而是详细告诉模型什么时候用这个工具、怎么用、用完之后期望什么结果、出错了怎么处理。

这些描述本身就是精心调优的提示词工程。换句话说，Anthropic的工程师花了大量时间做的事情，不是"怎么调用API"，而是"怎么在有限的上下文窗口里，让模型尽可能聪明地工作"。

记忆系统

泄露代码揭示了一个三层记忆架构：常驻上下文的轻量索引文件（MEMORY.md，每行约150字符），按需加载的主题知识文件，以及永远不会整体回读的原始对话记录——需要时只通过grep检索特定标识符。

这解决的是一个很朴实的问题：上下文窗口是有限的，你不能把所有历史对话都塞进去。热数据常驻，温数据按需，冷数据只留索引。思路不复杂，但能做到这个精细度需要大量工程打磨。



rednote ID: 1064397050

250,000次浪费的API调用

代码注释里有一段直白的内部记录：2026年3月10日，工程师BQ发现有1279个会话在单次对话中出现了50次以上的连续失败，最极端的案例高达3272次，导致每天全球浪费约25万次API调用。修复方案是三行代码：连续失败超过3次就停止重试。

有时候好的工程是知道什么时候该放弃。这条注释被很多人截图，因为它展示了一种少见的坦诚——把自己的bug和损耗写进代码注释里，明明白白。

四、那些还没上线的功能

源码里有44个未激活的feature flag，每一个都是一扇通往产品未来的窗户。

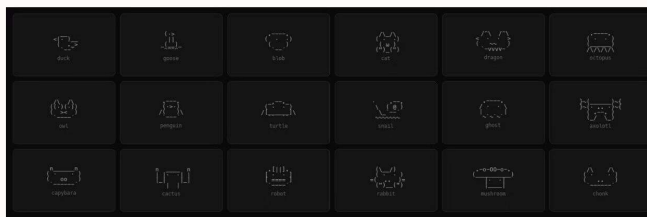
KAIROS：以古希腊语"恰当时机"命名，在源码中被提及超过150次。这是一个自主守护进程模式，让Claude Code作为always-on后台代理持续运行。更有意思的是它的"autoDream"逻辑——在用户空闲时，代理会进行记忆整合，合并零散观察，消除逻辑矛盾，把模糊洞察转化为确定性事实。



rednote ID: 1064397050

这个方向的意义不在技术细节，而在产品形态的转变：从"你问我答"的工具，到"持续理解你的项目、主动发现问题"的协作者。目前市场上的AI编程工具几乎全部是响应式的，KAIROS指向的是下一代。

BUDDY：一个拓麻歌子风格的终端宠物，有18个物种和稀有度等级。用户的物种不是随机的，而是用userId的哈希值确定性生成——同一个用户永远孵出同一个伙伴。代码注释显示原定4月1日到7日开始预热，5月全面上线。



ULTRAPLAN：30分钟远程规划会话模式。

VOICE_MODE：语音交互。

COORDINATOR MODE：多代理协调模式。

还有一个叫ANTI_DISTILLATION_CC的机制：当开启时，Claude Code会在API请求里加入虚假工具定义，专门污染那些试图录制API流量

的AI模型。

模型的人的训练数据。这个细节在Hacker News上引发了长时间讨论，有人觉得这是正当的自我保护，也有人觉得这触碰了某种边界。

五、"Undercover模式": Claude在偷偷给开源项目贡献代码

这是整次泄露里最让人不安的发现之一。

源码里有一段系统提示词，明确写着：

"你正在UNDERCOVER模式下运行……你的commit信息不能包含任何Anthropic内部信息。不要暴露身份。"

具体来说：commit信息里不能出现"Claude Code"，不能出现"Generated with Claude"，不能有"Co-Authored-By: Claude"之类的标注。

Hacker News的评论区对这个发现的反应比frustration regex更激烈。问题不在于AI是否应该参与开源项目的贡献——它当然可以——而在于这种参与是在刻意隐瞒身份的前提下进行的。AI写的代码，以人类的名义提交，进入公共代码库，成为其他人信任的工程基础设施。



rednote ID: 1064397050

有人指出，这和更早的争议直接相关：Anthropic 十天前刚对OpenCode发出法律威胁，因为后者用Claude Code的内部API以订阅价格访问Opus，而不是按token付费。一边用法律手段维护自己的权益边界，一边让AI悄悄在别人的开源项目里留下无法追溯的痕迹——这种对比，让很多开发者感到不舒服。

Anthropic没有就Undercover模式单独发表声明。

六、这次泄露是怎么发生的，以及为什么这不是第一次

从技术层面，这次泄露的原因极其简单：有人在发布npm包时，忘记把source map文件加进.npmignore。

Claude Code基于Bun构建——Anthropic在2025年底收购了Bun的团队。Bun默认生成source map，这是它的正常行为。问题是，没有人在发布配置里把这个文件排除掉。一个本该只存在于开发环境的59.8MB调试文件，就这样随着正式版本进了npm仓库。

更讽刺的是，这不是第一次。一位开发者在DEV

Community上发文指出，同样的问题早在2025年2月就发生过一次。那次显然没有引起足够重视，没有建立系统性的检查机制，于是一年后同样的错误再次发生，这次规模更大，影响更广。

还有一个巧合让人忍不住多想：泄露发生在3月31日——愚人节前一天。BUDDY系统的源码注释里，原定的预热时间是4月1日到7日。有人在推特上认真分析，这究竟是一次意外，还是一次精心设计的"意外营销"——毕竟这次泄露为Anthropic带来了无数开发者的自发传播，而Claude Code在2026年前三个月的年化收入刚刚达到25亿美元，是同期OpenAI同类产品的2.5倍。

Anthropic对外的官方表态是：人为失误，不涉及安全漏洞，用户数据未受影响。

写在最后

这次泄露最终暴露的，是一个关于AI编程工具的基本判断：它没有黑魔法。

51万行代码里，有精心调优的工具描述，有三层记忆管理，有frustration regex，有250,000次浪费API调用的坦诚注释，有还没来得及上线的终端宠

rednote ID: 1064397050

物，有悄悄给开源项目贡献代码的undercover模式。每一个细节都来自真实的工程决策，每一个决策背后都有具体的成本和权衡。

对开发者来说，这份源码是一本难得的教材——不是因为可以直接复用，而是因为它展示了一个生产级AI工具在细节层面是怎么被打磨出来的。

对用户来说，这次泄露意味着：你和Claude之间的对话，比你以为的更不对称。它在观察你。它知道你什么时候在发火。只是它没有告诉你。

那段frustration regex还在运行着。无论这篇文章发出去之后Claude Code改没改那段代码，那种工程直觉——大模型是贵的，够用就好，简单的事情用简单的方法解决——不会变。



rednote ID: 1064397050